Prescription Medicine Recognition - DoseBotV2

Goal: Train a CNN to classify handwritten prescription medicine names (78 classes), then deploy to HuggingFace Spaces.

Dataset: Doctor's Handwritten Prescription BD Dataset (Kaggle)

Cell 1: Install Dependencies

```
!pip install tensorflow pandas numpy scikit-learn matplotlib seaborn opencv-python-headless Pillow
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.20.0)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: opencv-python-headless in /usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages (11.3.0)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorfl
Requirement already satisfied: google_pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt_einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (26.2)
Requirement already satisfied: protobuf>=5.28.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (5.29.6)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing_extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.2.1)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.81.0)
Requirement already satisfied: tensorboard~=2.20.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.20.0)
Requirement already satisfied: keras>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.13.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.16.0)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.4)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensor
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (0.19.1)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tens
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tens
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorf
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensor
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorf
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1->tensorboa
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.10.0->t
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.10.0-
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->kera
```

```
import os
import json

# 1. Setup Kaggle API Credentials
# Replace 'YOUR_KAGGLE_USERNAME' with your actual Kaggle username
# The key provided: a8e88ad667cc7a7320dad58db1e079bf
kaggle_credentials = {"username": "chanupahansaja", "key": "a8e88ad667cc7a7320dad58db1e079bf"}

# Ensure the .kaggle directory exists
os.makedirs('/root/.kaggle', exist_ok=True)
with open('/root/.kaggle/kaggle.json', 'w') as f:
    json.dump(kaggle_credentials, f)

# Set permissions
!chmod 600 /root/.kaggle/kaggle.json
```

```
# 2. Download and extract the dataset
# The script expects data to be in /content/data
os.makedirs('/content/data', exist_ok=True)

print("Downloading dataset...")
!kaggle datasets download -d mamun1113/doctors-handwritten-prescription-bd-dataset -p /content/data

print("Extracting dataset...")
import zipfile
zip_path = '/content/data/doctors-handwritten-prescription-bd-dataset.zip'
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall('/content/data')

# Clean up the zip file
os.remove(zip_path)

print("Done! Dataset is now available in /content/data")
```

```
Downloading dataset...
Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.2), please consider upgrading to the latest ve
Dataset URL: https://www.kaggle.com/datasets/mamun1113/doctors-handwritten-prescription-bd-dataset
License(s): ODbL-1.0
Downloading doctors-handwritten-prescription-bd-dataset.zip to /content/data
100% 19.1M/19.1M [00:00<00:00, 94.4MB/s]

Extracting dataset...
Done! Dataset is now available in /content/data
```

Cell 2: Import Libraries

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import cv2
import pickle
import json
import warnings
warnings.filterwarnings('ignore')

%matplotlib inline

from PIL import Image
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.utils.class_weight import compute_class_weight
import tensorflow as tf
from tensorflow.keras.preprocessing.image import img_to_array, load_img, ImageDataGenerator
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (Conv2D, MaxPooling2D, Flatten, Dense,
                                     Dropout, BatchNormalization, GlobalAveragePooling2D)
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, ModelCheckpoint

print("TensorFlow version:", tf.__version__)
print("GPUs Available:", len(tf.config.list_physical_devices('GPU')))
```

```
TensorFlow version: 2.20.0
GPUs Available: 1
```

Cell 3: Configure Dataset Paths

```
# Auto-discover the dataset folder name (handles special unicode apostrophe)
# NOTE: Make sure you run Jupyter from the prescription-recognition folder,
#       or update NOTEBOOK_DIR below to the absolute path of this notebook.
NOTEBOOK_DIR = os.getcwd()  # Works when Jupyter is started from the notebook's folder
DATA_DIR = os.path.join(NOTEBOOK_DIR, "data")

print(f"Working directory: {NOTEBOOK_DIR}")
print(f"Data directory:    {DATA_DIR}")
print(f"Data dir exists:   {os.path.exists(DATA_DIR)}")

if not os.path.exists(DATA_DIR):
    raise FileNotFoundError(
        f"Data folder not found at {DATA_DIR}\n"
```

```
        f"Please either:\n"
        f"  1. Start Jupyter from: d:\\Logee Sir Project\\ML\\model\\prescription-recognition\n"
        f"  2. Or set NOTEBOOK_DIR above to the full path of the notebook folder"
    )

dataset_folder = os.listdir(DATA_DIR)[0]
BASE = os.path.join(DATA_DIR, dataset_folder)

TRAIN_IMG = os.path.join(BASE, "Training", "training_words")
TRAIN_CSV = os.path.join(BASE, "Training", "training_labels.csv")
TEST_IMG  = os.path.join(BASE, "Testing", "testing_words")
TEST_CSV  = os.path.join(BASE, "Testing", "testing_labels.csv")
VAL_IMG   = os.path.join(BASE, "Validation", "validation_words")
VAL_CSV   = os.path.join(BASE, "Validation", "validation_labels.csv")

# Handwritten words are wide rectangles, NOT squares. Using a square input
# (e.g. 128x128) warps the letters and destroys their morphology. We use a
# wide rectangular canvas and pad to preserve aspect ratio (see Cell 6).
IMG_W = 256   # width  (long edge - words are wide)
IMG_H = 64    # height (short edge)
NUM_CLASSES = 78
BATCH_SIZE = 32
EPOCHS = 60

print(f"Dataset: {dataset_folder}")
print(f"Input shape: {IMG_H}x{IMG_W} (HxW), aspect-ratio preserved with padding")
print(f"Train images: {len(os.listdir(TRAIN_IMG))}")
print(f"Test images:  {len(os.listdir(TEST_IMG))}")
print(f"Val images:   {len(os.listdir(VAL_IMG))}")
```

```
Working directory: /content
Data directory:    /content/data
Data dir exists:   True
Dataset: Doctor’s Handwritten Prescription BD dataset
Input shape: 64x256 (HxW), aspect-ratio preserved with padding
Train images: 3120
Test images:  780
Val images:   780
```

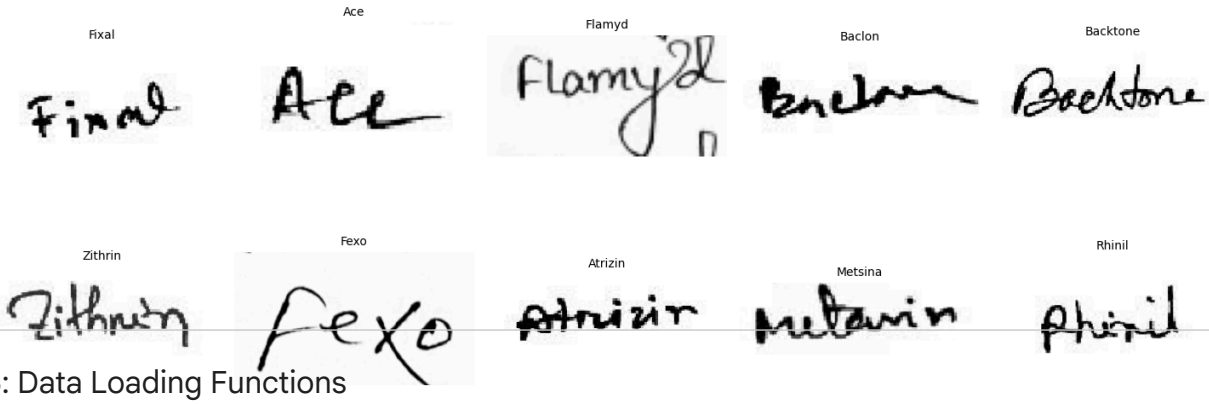
Cell 4: Load & Explore Data

```
train_df = pd.read_csv(TRAIN_CSV)
test_df  = pd.read_csv(TEST_CSV)
val_df   = pd.read_csv(VAL_CSV)

print("Training data shape:", train_df.shape)
print("Columns:", train_df.columns.tolist())
print("\nSample rows:")
print(train_df.head(10))
print(f"\nUnique medicines: {train_df['MEDICINE_NAME'].nunique()}")
print(f"\nClass distribution:\n{train_df['MEDICINE_NAME'].value_counts()}")
```

```
Training data shape: (3120, 3)
Columns: ['IMAGE', 'MEDICINE_NAME', 'GENERIC_NAME']

Sample rows:
   IMAGE MEDICINE_NAME GENERIC_NAME
0  0.png         Aceta  Paracetamol
1  1.png         Aceta  Paracetamol
2  2.png         Aceta  Paracetamol
3  3.png         Aceta  Paracetamol
4  4.png         Aceta  Paracetamol
5  5.png         Aceta  Paracetamol
6  6.png         Aceta  Paracetamol
7  7.png         Aceta  Paracetamol
8  8.png         Aceta  Paracetamol
9  9.png         Aceta  Paracetamol

Unique medicines: 78

Class distribution:
MEDICINE_NAME
Aceta       40
Ace         40
Alatrol     40
Amodis      40
Atrizin     40
            ..
Telfast     40
Tridosil    40
Trilock     40
Vifas       40
Zithrin     40
Name: count, Length: 78, dtype: int64
```

Cell 5: Visualize Sample Images

```
fig, axes = plt.subplots(3, 5, figsize=(15, 9))
fig.suptitle("Sample Prescription Handwriting", fontsize=16, fontweight='bold')

sample = train_df.groupby('MEDICINE_NAME').first().reset_index().sample(15, random_state=42)
for idx, (ax, (_, row)) in enumerate(zip(axes.flat, sample.iterrows())):
    img = cv2.imread(os.path.join(TRAIN_IMG, row['IMAGE']))
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    ax.imshow(img)
    ax.set_title(row['MEDICINE_NAME'], fontsize=10)
    ax.axis('off')

plt.tight_layout()
plt.savefig("sample_images.png", dpi=100)
plt.show()
```



Cell 6: Data Loading Functions

```
def resize_with_padding(img, target_w=IMG_W, target_h=IMG_H, pad_color=(255, 255, 255)):
    """Resize a PIL image to (target_w, target_h) WITHOUT distorting it.

    The image is scaled so its longest edge fits, then centered on a white
    canvas. This preserves the natural wide-and-short shape of handwritten
    words instead of squashing them into a square.
    """
    w, h = img.size
    scale = min(target_w / w, target_h / h)
    new_w, new_h = max(1, int(round(w * scale))), max(1, int(round(h * scale)))
    img = img.resize((new_w, new_h), Image.BILINEAR)
    canvas = Image.new("RGB", (target_w, target_h), pad_color)
    canvas.paste(img, ((target_w - new_w) // 2, (target_h - new_h) // 2))
    return canvas


def load_images_and_labels(img_folder, csv_path, target_w=IMG_W, target_h=IMG_H):
    """Load images and labels from folder + CSV (aspect-ratio preserved)."""
    df = pd.read_csv(csv_path)
    images, labels = [], []
    skipped = 0

    for _, row in df.iterrows():
        img_path = os.path.join(img_folder, row['IMAGE'])
        if not os.path.exists(img_path):
            skipped += 1
            continue
        img = Image.open(img_path).convert("RGB")
        img = resize_with_padding(img, target_w, target_h)
        images.append(np.array(img, dtype="float32"))
        labels.append(row['MEDICINE_NAME'])

    if skipped > 0:
        print(f"  Warning: skipped {skipped} missing images")

    images = np.array(images, dtype="float32") / 255.0  # shape: (N, IMG_H, IMG_W, 3)
    labels = np.array(labels)
    return images, labels

print("Loading training data...")
X_train, y_train_raw = load_images_and_labels(TRAIN_IMG, TRAIN_CSV)
print(f"  Shape: {X_train.shape}, Labels: {len(y_train_raw)}")
```

```
print("Loading validation data...")
X_val, y_val_raw = load_images_and_labels(VAL_IMG, VAL_CSV)
print(f"  Shape: {X_val.shape}, Labels: {len(y_val_raw)}")

print("Loading test data...")
X_test, y_test_raw = load_images_and_labels(TEST_IMG, TEST_CSV)
print(f"  Shape: {X_test.shape}, Labels: {len(y_test_raw)}")
```

```
Loading training data...
  Shape: (3120, 64, 256, 3), Labels: 3120
Loading validation data...
  Shape: (780, 64, 256, 3), Labels: 780
Loading test data...
  Shape: (780, 64, 256, 3), Labels: 780
```

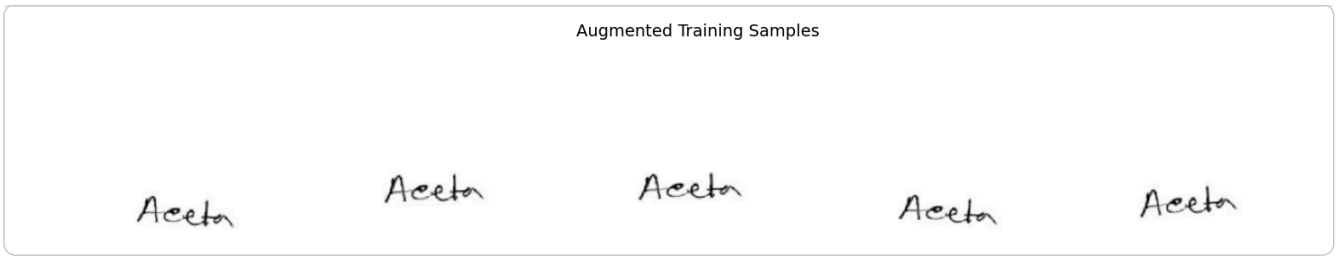
Cell 7: Encode Labels

```
# Fit label encoder on ALL labels combined to ensure consistency
all_labels = np.concatenate([y_train_raw, y_val_raw, y_test_raw])
label_encoder = LabelEncoder()
label_encoder.fit(all_labels)

y_train = to_categorical(label_encoder.transform(y_train_raw), NUM_CLASSES)
y_val   = to_categorical(label_encoder.transform(y_val_raw),   NUM_CLASSES)
y_test  = to_categorical(label_encoder.transform(y_test_raw),  NUM_CLASSES)

print(f"Classes ({len(label_encoder.classes_)}):")
print(list(label_encoder.classes_))

# Save label encoder for deployment
with open("label_encoder.pkl", "wb") as f:
    pickle.dump(label_encoder, f)

# Also save as JSON for easier use
label_map = {int(i): name for i, name in enumerate(label_encoder.classes_)}
with open("label_map.json", "w") as f:
    json.dump(label_map, f, indent=2)

print("\nLabel encoder saved!")
```

```
Classes (78):
[np.str_('Ace'), np.str_('Aceta'), np.str_('Alatrol'), np.str_('Amodis'), np.str_('Atrizin'), np.str_('Axodin'), np.str_('Az

Label encoder saved!
```

Cell 8: Data Augmentation

```
# Mild augmentation only. Aggressive rotation/shear/brightness on top of heavy
# dropout was a major cause of underfitting (the model could never even fit the
# training data). Keep jitter small so letters stay legible.
datagen = ImageDataGenerator(
    rotation_range=5,
    zoom_range=0.05,
    width_shift_range=0.05,
    height_shift_range=0.05,
    fill_mode='nearest',
    cval=255  # pad with white to match resize_with_padding
)

datagen.fit(X_train)

# Visualize augmented samples
fig, axes = plt.subplots(2, 5, figsize=(15, 6))
fig.suptitle("Augmented Training Samples", fontsize=14)
sample_img = X_train[0:1]
for ax in axes.flat:
    aug_img = datagen.flow(sample_img, batch_size=1)[0][0]
    ax.imshow(np.clip(aug_img, 0, 1))
    ax.axis('off')
plt.tight_layout()
plt.savefig("augmented_samples.png", dpi=100)
plt.show()
```



Cell 9: Build Improved CNN Model

```
def build_model(input_shape=(IMG_H, IMG_W, 3), num_classes=NUM_CLASSES):
    """CNN feature extractor + light classifier.

    Regularization was drastically reduced vs. the original: the old model had
    Dropout(0.25) after EVERY conv block plus two Dropout(0.5) dense layers,
    which made it impossible to even fit the training set (<6% train acc). We
    now rely on BatchNorm + a single small dropout. Goal: let it overfit first,
    then add regularization back only if val/train gap becomes large.
    """
    model = Sequential([
        # Block 1
        Conv2D(32, (3, 3), activation='relu', padding='same', input_shape=input_shape),
        BatchNormalization(),
        Conv2D(32, (3, 3), activation='relu', padding='same'),
        BatchNormalization(),
        MaxPooling2D(pool_size=(2, 2)),

        # Block 2
        Conv2D(64, (3, 3), activation='relu', padding='same'),
        BatchNormalization(),
        Conv2D(64, (3, 3), activation='relu', padding='same'),
        BatchNormalization(),
        MaxPooling2D(pool_size=(2, 2)),

        # Block 3
        Conv2D(128, (3, 3), activation='relu', padding='same'),
        BatchNormalization(),
        Conv2D(128, (3, 3), activation='relu', padding='same'),
        BatchNormalization(),
        MaxPooling2D(pool_size=(2, 2)),

        # Block 4
        Conv2D(256, (3, 3), activation='relu', padding='same'),
        BatchNormalization(),
        Conv2D(256, (3, 3), activation='relu', padding='same'),
        BatchNormalization(),
        MaxPooling2D(pool_size=(2, 2)),

        # Classifier (single dense layer, light dropout)
        GlobalAveragePooling2D(),
        Dense(512, activation='relu'),
        BatchNormalization(),
        Dropout(0.2),
        Dense(num_classes, activation='softmax')
    ])
    return model

model = build_model()
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 64, 256, 32)	896
batch_normalization (BatchNormalization)	(None, 64, 256, 32)	128
conv2d_1 (Conv2D)	(None, 64, 256, 32)	9,248
batch_normalization_1 (BatchNormalization)	(None, 64, 256, 32)	128
max_pooling2d (MaxPooling2D)	(None, 32, 128, 32)	0
conv2d_2 (Conv2D)	(None, 32, 128, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 32, 128, 64)	256
conv2d_3 (Conv2D)	(None, 32, 128, 64)	36,928
batch_normalization_3 (BatchNormalization)	(None, 32, 128, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 16, 64, 64)	0
conv2d_4 (Conv2D)	(None, 16, 64, 128)	73,856
batch_normalization_4 (BatchNormalization)	(None, 16, 64, 128)	512
conv2d_5 (Conv2D)	(None, 16, 64, 128)	147,584
batch_normalization_5 (BatchNormalization)	(None, 16, 64, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 8, 32, 128)	0
conv2d_6 (Conv2D)	(None, 8, 32, 256)	295,168
batch_normalization_6 (BatchNormalization)	(None, 8, 32, 256)	1,024
conv2d_7 (Conv2D)	(None, 8, 32, 256)	590,080
batch_normalization_7 (BatchNormalization)	(None, 8, 32, 256)	1,024
max_pooling2d_3 (MaxPooling2D)	(None, 4, 16, 256)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 256)	0
dense (Dense)	(None, 512)	131,584
batch_normalization_8 (BatchNormalization)	(None, 512)	2,048
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 78)	40,014

Cell 10: Compile & Train

```
# Lower LR (1e-3 -> 1e-4): the old rate caused the loss spikes / unstable
# accuracy and let the optimizer collapse onto the majority class.
model.compile(
    optimizer=Adam(learning_rate=1e-4),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Class weights: counter imbalance so the model can't win by always guessing
# the majority class (the "always predicts Flexibac" failure mode).
y_train_int = label_encoder.transform(y_train_raw)
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.arange(NUM_CLASSES),
    y=y_train_int
)
class_weight_dict = dict(enumerate(class_weights))
print(f"Class weights range: {class_weights.min():.2f} - {class_weights.max():.2f}")

callbacks = [
    EarlyStopping(monitor='val_accuracy', patience=12, restore_best_weights=True, verbose=1),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=5, min_lr=1e-6, verbose=1),
    ModelCheckpoint('best_model.keras', monitor='val_accuracy', save_best_only=True, verbose=1)
]
```

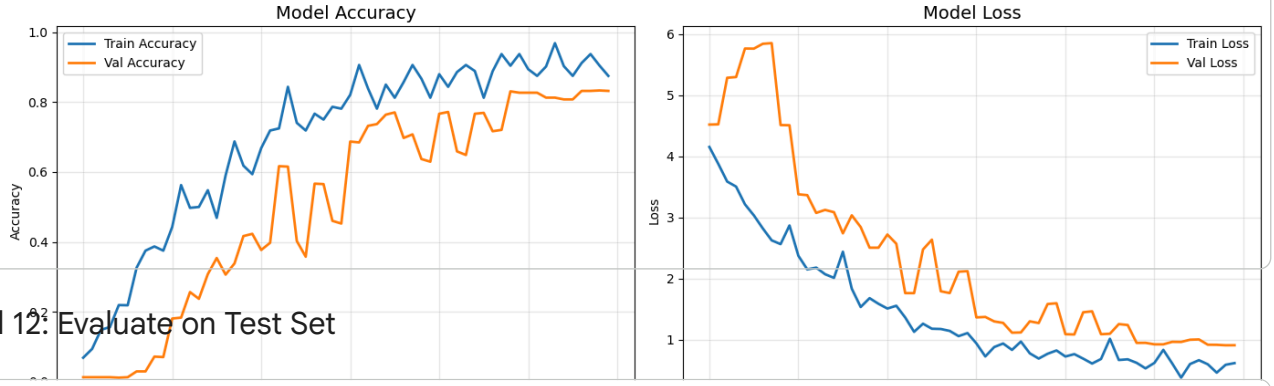
```
history = model.fit(
    datagen.flow(X_train, y_train, batch_size=BATCH_SIZE),
    validation_data=(X_val, y_val),
    epochs=EPOCHS,
    callbacks=callbacks,
    class_weight=class_weight_dict,
    steps_per_epoch=len(X_train) // BATCH_SIZE
)
```

```
Class weights range: 1.00 - 1.00
Epoch 1/60
97/97 ━━━━━━━━━━━━━━━━━━━━ 0s 267ms/step - accuracy: 0.0482 - loss: 4.3930
Epoch 1: val_accuracy improved from None to 0.01282, saving model to best_model.keras

Epoch 1: finished saving model to best_model.keras
97/97 ━━━━━━━━━━━━━━━━━━━━ 54s 327ms/step - accuracy: 0.0687 - loss: 4.1536 - val_accuracy: 0.0128 - val_loss: 4.5196 - le
Epoch 2/60
 1/97 ━━━━━━━━━━━━━━━━━━━━ 4s 48ms/step - accuracy: 0.0938 - loss: 3.8771
Epoch 2: val_accuracy did not improve from 0.01282
97/97 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.0938 - loss: 3.8771 - val_accuracy: 0.0128 - val_loss: 4.5236 - learn
Epoch 3/60
97/97 ━━━━━━━━━━━━━━━━━━━━ 0s 150ms/step - accuracy: 0.1390 - loss: 3.6729
Epoch 3: val_accuracy did not improve from 0.01282
97/97 ━━━━━━━━━━━━━━━━━━━━ 15s 156ms/step - accuracy: 0.1486 - loss: 3.5873 - val_accuracy: 0.0128 - val_loss: 5.2854 - le
Epoch 4/60
 1/97 ━━━━━━━━━━━━━━━━━━━━ 5s 55ms/step - accuracy: 0.1562 - loss: 3.5050
Epoch 4: val_accuracy did not improve from 0.01282
97/97 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.1562 - loss: 3.5050 - val_accuracy: 0.0128 - val_loss: 5.2972 - learn
Epoch 5/60
97/97 ━━━━━━━━━━━━━━━━━━━━ 0s 149ms/step - accuracy: 0.2176 - loss: 3.2413
Epoch 5: val_accuracy did not improve from 0.01282
97/97 ━━━━━━━━━━━━━━━━━━━━ 15s 154ms/step - accuracy: 0.2192 - loss: 3.2142 - val_accuracy: 0.0115 - val_loss: 5.7639 - le
Epoch 6/60
 1/97 ━━━━━━━━━━━━━━━━━━━━ 4s 49ms/step - accuracy: 0.2188 - loss: 3.0338
Epoch 6: ReduceLROnPlateau reducing learning rate to 4.999999873689376e-05.

Epoch 6: val_accuracy did not improve from 0.01282
97/97 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.2188 - loss: 3.0338 - val_accuracy: 0.0128 - val_loss: 5.7622 - learn
Epoch 7/60
97/97 ━━━━━━━━━━━━━━━━━━━━ 0s 146ms/step - accuracy: 0.2965 - loss: 2.9143
Epoch 7: val_accuracy improved from 0.01282 to 0.02949, saving model to best_model.keras

Epoch 7: finished saving model to best_model.keras
97/97 ━━━━━━━━━━━━━━━━━━━━ 15s 153ms/step - accuracy: 0.3261 - loss: 2.8213 - val_accuracy: 0.0295 - val_loss: 5.8411 - le
Epoch 8/60
 1/97 ━━━━━━━━━━━━━━━━━━━━ 4s 46ms/step - accuracy: 0.3750 - loss: 2.6255
Epoch 8: val_accuracy did not improve from 0.02949
97/97 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.3750 - loss: 2.6255 - val_accuracy: 0.0295 - val_loss: 5.8519 - learn
Epoch 9/60
97/97 ━━━━━━━━━━━━━━━━━━━━ 0s 147ms/step - accuracy: 0.3775 - loss: 2.6134
Epoch 9: val_accuracy improved from 0.02949 to 0.07179, saving model to best_model.keras

Epoch 9: finished saving model to best_model.keras
97/97 ━━━━━━━━━━━━━━━━━━━━ 15s 153ms/step - accuracy: 0.3870 - loss: 2.5642 - val_accuracy: 0.0718 - val_loss: 4.5116 - le
Epoch 10/60
 1/97 ━━━━━━━━━━━━━━━━━━━━ 4s 48ms/step - accuracy: 0.3750 - loss: 2.8689
Epoch 10: val_accuracy did not improve from 0.07179
97/97 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.3750 - loss: 2.8689 - val_accuracy: 0.0705 - val_loss: 4.5086 - learn
Epoch 11/60
97/97 ━━━━━━━━━━━━━━━━━━━━ 0s 148ms/step - accuracy: 0.4393 - loss: 2.4021
Epoch 11: val_accuracy improved from 0.07179 to 0.18077, saving model to best_model.keras

Epoch 11: finished saving model to best_model.keras
97/97 ━━━━━━━━━━━━━━━━━━━━ 15s 155ms/step - accuracy: 0.4427 - loss: 2.3727 - val_accuracy: 0.1808 - val_loss: 3.3790 - le
Epoch 12/60
 1/97 ━━━━━━━━━━━━━━━━━━━━ 4s 47ms/step - accuracy: 0.5625 - loss: 2.1501
```

Cell 11: Plot Training History

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

ax1.plot(history.history['accuracy'], label='Train Accuracy', linewidth=2)
ax1.plot(history.history['val_accuracy'], label='Val Accuracy', linewidth=2)
ax1.set_title('Model Accuracy', fontsize=14)
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Accuracy')
ax1.legend()
ax1.grid(True, alpha=0.3)

ax2.plot(history.history['loss'], label='Train Loss', linewidth=2)
ax2.plot(history.history['val_loss'], label='Val Loss', linewidth=2)
ax2.set_title('Model Loss', fontsize=14)
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Loss')
```

```
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("training_history.png", dpi=100)
plt.show()
```



Cell 12: Evaluate on Test Set

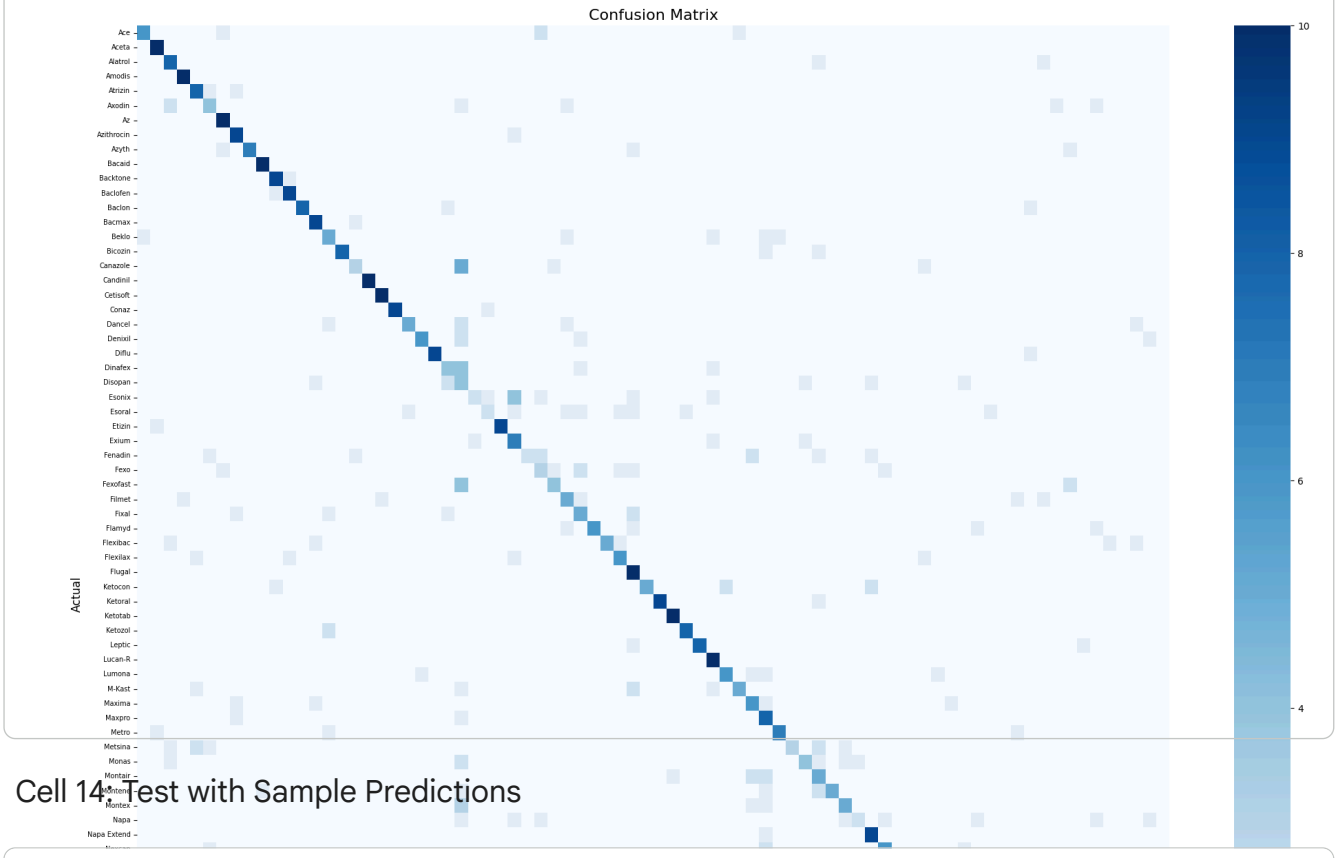
```
test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=1)
print(f"\n{'='*50}")
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")
print(f"Test Loss: {test_loss:.4f}")
print(f"{'='*50}")

# Classification report
predictions = model.predict(X_test)
pred_classes = np.argmax(predictions, axis=1)
true_classes = np.argmax(y_test, axis=1)

print("\nClassification Report:")
print(classification_report(true_classes, pred_classes,
                           target_names=label_encoder.classes_))
```

```
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - accuracy: 0.6397 - loss: 1.6981

==================================================
Test Accuracy: 63.97%
Test Loss: 1.6981
==================================================
25/25 ━━━━━━━━━━━━━━━━━━━━ 2s 48ms/step

Classification Report:
              precision    recall  f1-score   support

         Ace       0.86      0.60      0.71        10
       Aceta       0.77      1.00      0.87        10
     Alatrol       0.57      0.80      0.67        10
      Amodis       0.77      1.00      0.87        10
     Atrizin       0.53      0.80      0.64        10
      Axodin       0.40      0.40      0.40        10
          Az       0.77      1.00      0.87        10
  Azithrocin       0.69      0.90      0.78        10
       Azyth       0.88      0.70      0.78        10
      Bacaid       0.91      1.00      0.95        10
    Backtone       0.82      0.90      0.86        10
    Baclofen       0.69      0.90      0.78        10
      Baclon       1.00      0.80      0.89        10
      Bacmax       0.75      0.90      0.82        10
       Beklo       0.50      0.50      0.50        10
     Bicozin       1.00      0.80      0.89        10
    Canazole       0.50      0.30      0.38        10
    Candinil       0.91      1.00      0.95        10
    Cetisoft       0.91      1.00      0.95        10
       Conaz       0.75      0.90      0.82        10
      Dancel       0.83      0.50      0.62        10
     Denixil       0.75      0.60      0.67        10
       Diflu       0.75      0.90      0.82        10
     Dinafex       0.36      0.40      0.38        10
     Disopan       0.11      0.40      0.17        10
      Esonix       0.67      0.20      0.31        10
      Esoral       0.50      0.20      0.29        10
      Etizin       0.75      0.90      0.82        10
       Exium       0.47      0.70      0.56        10
     Fenadin       1.00      0.20      0.33        10
        Fexo       0.33      0.30      0.32        10
    Fexofast       0.50      0.40      0.44        10
      Filmet       0.50      0.50      0.50        10
       Fixal       0.36      0.50      0.42        10
      Flamyd       0.86      0.60      0.71        10
    Flexibac       1.00      0.50      0.67        10
    Flexilax       0.67      0.60      0.63        10
      Flugal       0.43      1.00      0.61        10
```

```
Ketocon       0.83      0.50      0.62        10
Ketoral       0.82      0.90      0.86        10
Ketotab       0.91      1.00      0.95        10
Ketozol       0.89      0.80      0.84        10
 Leptic       0.73      0.80      0.76        10
Lucan-R       0.62      1.00      0.77        10
 Lumona       0.67      0.60      0.63        10
 M-Kast       0.83      0.50      0.62        10
 Maxima       0.50      0.60      0.55        10
```

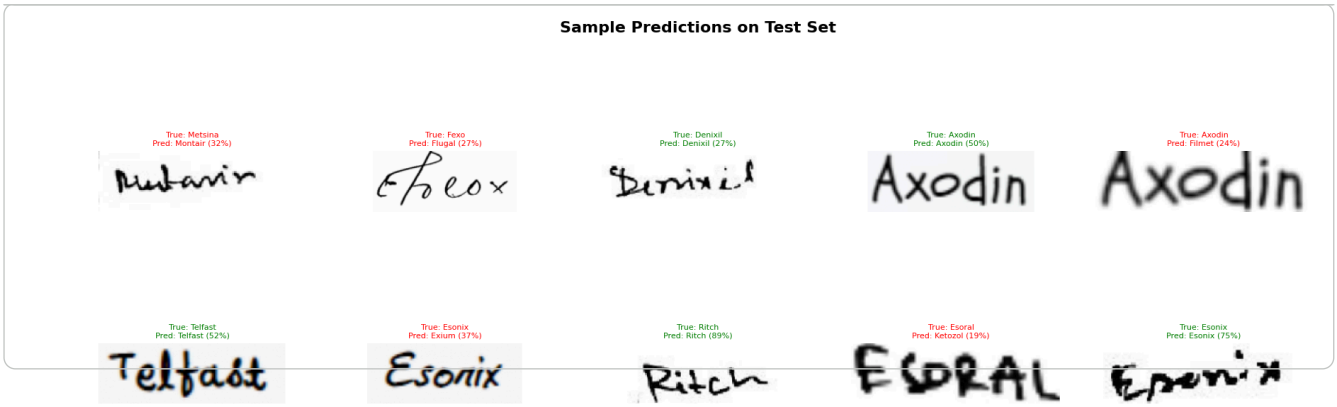
Cell 13: Confusion Matrix

```
cm = confusion_matrix(true_classes, pred_classes)
plt.figure(figsize=(20, 18))
sns.heatmap(cm, annot=False, cmap='Blues',
            xticklabels=label_encoder.classes_,
            yticklabels=label_encoder.classes_)
plt.title('Confusion Matrix', fontsize=16)
plt.xlabel('Predicted', fontsize=12)
plt.ylabel('Actual', fontsize=12)
plt.xticks(rotation=90, fontsize=7)
plt.yticks(rotation=0, fontsize=7)
plt.tight_layout()
plt.savefig("confusion_matrix.png", dpi=100)
plt.show()
```



Cell 14: Test with Sample Predictions

```
fig, axes = plt.subplots(3, 5, figsize=(18, 10))
fig.suptitle("Sample Predictions on Test Set", fontsize=16, fontweight='bold')

indices = np.random.choice(len(X_test), 15, replace=False)
for ax, idx in zip(axes.flat, indices):
    img = X_test[idx]
    true_label = label_encoder.classes_[true_classes[idx]]
    pred_label = label_encoder.classes_[pred_classes[idx]]
    confidence = predictions[idx][pred_classes[idx]] * 100

    ax.imshow(img)
    color = 'green' if true_label == pred_label else 'red'
    ax.set_title(f"True: {true_label}\nPred: {pred_label} ({confidence:.0f}%)",
                 fontsize=8, color=color)
    ax.axis('off')

plt.tight_layout()
plt.savefig("sample_predictions.png", dpi=100)
plt.show()
```



Cell 15: Save Final Model for Deployment

```
# Save in multiple formats
model.save("prescription_model.keras")
model.save("prescription_model.h5")

# Save model config
model_info = {
    "img_w": IMG_W,
    "img_h": IMG_H,
    "num_classes": NUM_CLASSES,
    "classes": list(label_encoder.classes_),
    "test_accuracy": float(test_accuracy),
    "test_loss": float(test_loss)
}
with open("model_info.json", "w") as f:
    json.dump(model_info, f, indent=2)

print("Models saved:")
print("  - prescription_model.keras")
print("  - prescription_model.h5")
print("  - label_encoder.pkl")
print("  - label_map.json")
print("  - model_info.json")
print(f"\nFinal Test Accuracy: {test_accuracy*100:.2f}%")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file for
Models saved:
  - prescription_model.keras
  - prescription_model.h5
  - label_encoder.pkl
  - label_map.json
  - model_info.json

Final Test Accuracy: 63.97%
```

Cell 16: Create HuggingFace Deployment Files

The following cells generate the files needed for deployment.

```
# Generate app.py for HuggingFace Spaces
app_code = '''import gradio as gr
import tensorflow as tf
import numpy as np
import json
from PIL import Image

# Load model and label map
model = tf.keras.models.load_model("prescription_model.h5")

with open("label_map.json", "r") as f:
    label_map = json.load(f)

with open("model_info.json", "r") as f:
    model_info = json.load(f)

IMG_W = model_info["img_w"]
IMG_H = model_info["img_h"]


def resize_with_padding(img, target_w, target_h, pad_color=(255, 255, 255)):
    """Match the training preprocessing: scale to fit, then pad (no warping)."""
    w, h = img.size
```

```
    scale = min(target_w / w, target_h / h)
    new_w, new_h = max(1, int(round(w * scale))), max(1, int(round(h * scale)))
    img = img.resize((new_w, new_h), Image.BILINEAR)
    canvas = Image.new("RGB", (target_w, target_h), pad_color)
    canvas.paste(img, ((target_w - new_w) // 2, (target_h - new_h) // 2))
    return canvas


def predict_medicine(image):
    """Predict medicine name from handwritten prescription image."""
    if image is None:
        return {}, "Please upload an image"

    # Preprocess (aspect-ratio preserved, same as training)
    img = Image.fromarray(image).convert("RGB")
    img = resize_with_padding(img, IMG_W, IMG_H)
    img_array = np.array(img, dtype="float32") / 255.0
    img_array = np.expand_dims(img_array, axis=0)

    # Predict
    predictions = model.predict(img_array, verbose=0)[0]
    top_k = 5
    top_indices = predictions.argsort()[-top_k:][::-1]

    results = {}
    details = []
    for i, idx in enumerate(top_indices):
        label = label_map[str(idx)]
        conf = float(predictions[idx])
        results[label] = conf
        details.append(f"{i+1}. {label}: {conf*100:.1f}%")

    top_medicine = label_map[str(top_indices[0])]
    top_conf = predictions[top_indices[0]] * 100
    summary = f"**Predicted Medicine:** {top_medicine} ({top_conf:.1f}% confidence)"

    return results, summary

# Build Gradio interface
with gr.Blocks(
    title="DoseBotV2 - Prescription Recognition",
    theme=gr.themes.Soft(primary_hue="blue", secondary_hue="cyan")
) as demo:
    gr.Markdown("""
    # 🤖💊 DoseBotV2 - Prescription Medicine Recognition
    Upload a handwritten prescription image to identify the medicine name.
    Trained on 78 medicine classes from the Doctor\\'s Handwritten Prescription BD Dataset.
    """)

    with gr.Row():
        with gr.Column(scale=1):
            image_input = gr.Image(label="Upload Prescription Image", type="numpy")
            predict_btn = gr.Button("🔍 Identify Medicine", variant="primary", size="lg")
            gr.Examples(
                examples=[],
                inputs=image_input,
                label="Try these examples"
            )

        with gr.Column(scale=1):
            label_output = gr.Label(label="Prediction Confidence", num_top_classes=5)
            text_output = gr.Markdown(label="Result")

    predict_btn.click(
        fn=predict_medicine,
        inputs=image_input,
        outputs=[label_output, text_output]
    )

    gr.Markdown(f"""
    ---
    **Model Info:** CNN trained on {model_info['num_classes']} medicine classes |
    Test Accuracy: {model_info['test_accuracy']*100:.1f}%
    """)

demo.launch()
'''

with open("app.py", "w") as f:
    f.write(app_code)
print("Created: app.py")
```

```
# Generate requirements.txt
req = """tensorflow-cpu==2.16.1
gradio>=4.0.0
numpy
Pillow
"""
with open("requirements.txt", "w") as f:
    f.write(req)
print("Created: requirements.txt")

# Generate README.md for HF Space
readme = """---
title: DoseBotV2
emoji: 💊
colorFrom: blue
colorTo: cyan
sdk: gradio
sdk_version: 5.34.2
app_file: app.py
pinned: false
license: mit
---

# 🤖💊 DoseBotV2 - Prescription Medicine Recognition

Upload a handwritten prescription image to identify the medicine name.

## Model
- **Architecture:** CNN (4 conv blocks + BatchNorm + GlobalAvgPooling)
- **Input:** 256x64 RGB images (aspect-ratio preserved with padding)
- **Classes:** 78 medicine names
- **Dataset:** Doctor's Handwritten Prescription BD Dataset
"""
with open("README_HF.md", "w") as f:
    f.write(readme)
print("Created: README_HF.md")
print("\n✅ All deployment files generated!")
```

```
Created: app.py
Created: requirements.txt
Created: README_HF.md

✅ All deployment files generated!
```

Cell 17: Upload to HuggingFace (Run Manually)

```
# Install huggingface_hub
pip install huggingface_hub

# Login
huggingface-cli login

# Clone your space
git clone https://huggingface.co/spaces/Chanu2003/DoseBotV2
cd DoseBotV2

# Copy these files into the cloned repo:
#   - app.py
#   - requirements.txt
#   - README_HF.md -> rename to README.md
#   - prescription_model.h5
#   - label_map.json
#   - model_info.json

# Push to HuggingFace
git add .
git commit -m "Deploy DoseBotV2 prescription recognition model"
git push
```

Cell 18: Automated Upload to HuggingFace Spaces

This cell uses your provided token to create the space and upload all deployment files automatically.

```
import os
from huggingface_hub import HfApi, create_repo, login

# 1. Login with the provided token
# ⚠️ SECURITY WARNING: Please revoke this token in your HF Settings immediately after use!
HF_TOKEN = "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"
login(token=HF_TOKEN)

# 2. Define Space details
api = HfApi()
username = api.whoami()["name"]
repo_id = f"{username}/DoseBotV2"

print(f"Target Repository: {repo_id}")

# 3. Create the Space (if it doesn't exist)
try:
    create_repo(repo_id=repo_id, repo_type="space", space_sdk="gradio", exist_ok=True)
    print(f"Space {repo_id} ready.")
except Exception as e:
    print(f"Note: {e}")

# ========================================
# 🔧 NEW: Patch the README_HF.md file here
# ========================================
if os.path.exists("README_HF.md"):
    with open("README_HF.md", "r") as f:
        readme_content = f.read()

    # Replace the invalid color 'cyan' with an allowed color like 'green'
    readme_content = readme_content.replace("colorTo: cyan", "colorTo: green")

    with open("README_HF.md", "w") as f:
        f.write(readme_content)
    print("🔧 Patched README_HF.md successfully (changed cyan to green).")
else:
    print("⚠️ README_HF.md not found to patch.")
# ========================================

# 4. List of files to upload
files_to_upload = {
    "app.py": "app.py",
    "requirements.txt": "requirements.txt",
    "README_HF.md": "README.md",
    "prescription_model.h5": "prescription_model.h5",
    "label_map.json": "label_map.json",
    "model_info.json": "model_info.json"
}

# 5. Upload files
print("Uploading files to HuggingFace...")
for local_path, repo_path in files_to_upload.items():
    if os.path.exists(local_path):
        api.upload_file(
            path_or_fileobj=local_path,
            path_in_repo=repo_path,
            repo_id=repo_id,
            repo_type="space"
        )
        print(f"  - Uploaded {repo_path}")
    else:
        print(f"  - Warning: {local_path} not found, skipping.")

print(f"\n✅ Deployment complete! Check your space at: https://huggingface.co/spaces/{repo_id}")
```

```
Target Repository: Chanu2003/DoseBotV2
Space Chanu2003/DoseBotV2 ready.
🔧 Patched README_HF.md successfully (changed cyan to green).
Uploading files to HuggingFace...
No files have been modified since last commit. Skipping to prevent empty commit.
WARNING:huggingface_hub.hf_api:No files have been modified since last commit. Skipping to prevent empty commit.
No files have been modified since last commit. Skipping to prevent empty commit.
WARNING:huggingface_hub.hf_api:No files have been modified since last commit. Skipping to prevent empty commit.
  - Uploaded app.py
  - Uploaded requirements.txt
  - Uploaded README.md
Processing Files (1 / 1)      : 100%      16.3MB / 16.3MB, 1.51MB/s
New Data Upload               : 100%      16.3MB / 16.3MB, 1.51MB/s
  prescription_model.h5       : 100%                       16.3MB / 16.3MB
  - Uploaded prescription_model.h5
```